

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Математические основы интеллектуальных систем»
За 3 семестр

Тема: «Нахождение эйлеровых и гамильтоновых циклов в неориентированном графе»

Выполнил:
Студент ФЭИС
Группы ИИ-29
Игнатюк Д.А.

Проверил:
Козинский А.А.

Лабораторная работа №2

«Нахождение эйлеровых и гамильтоновых циклов в неориентированном графе»

Цель работы: освоить методы поиска эйлерова и гамильтонова циклов в неориентированном графе и научиться реализовывать их в виде программы на языке C.

Ход работы

Код

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
// ПРЕДСТАВЛЕНИЕ ГРАФА: СПИСКИ СМЕЖНОСТИ
typedef struct Edge {
    int v;
    struct Edge* next;
} Edge;
typedef struct {
    int n;
    int m;
    Edge** adj;
} Graph;
// Добавление неориентированного ребра
static void add_edge(Graph* g, int u, int v)
{
    Edge* p = (Edge*)malloc(sizeof(Edge));
    p->v = v; p->next = g->adj[u]; g->adj[u] = p;
    p = (Edge*)malloc(sizeof(Edge));
    p->v = u; p->next = g->adj[v]; g->adj[v] = p;
    g->m++;
}
// Чтение графа из файла
// Формат: первая строка — n; далее строки u v
static int read_graph(const char* fname, Graph* g)
{
    FILE* f = fopen(fname, "r");
    if (!f) { fprintf(stderr, "ne otkryt %s\n", fname); return 0; }
    char line[256];
    int n = 0;
    int header_ok = 0;
    while (fgets(line, sizeof(line), f)) {
        if (sscanf(line, "%d", &n) == 1) { header_ok = 1; break; }
    }
    if (!header_ok || n < 0) {
        fprintf(stderr, "%s: plokhoye n\n", fname);
        fclose(f); return 0;
    }
    g->n = n;
    g->m = 0;
    g->adj = (Edge**)calloc(n + 1, sizeof(Edge*));
    if (!g->adj) { fclose(f); return 0; }
    while (fgets(line, sizeof(line), f)) {
        int u = 0, v = 0;
        int got = sscanf(line, "%d %d", &u, &v);
        if (got < 2) continue;
        if (u < 1 || u > g->n || v < 1 || v > g->n) {
```

```

        fprintf(stderr, "%s: rebro (%d,%d) vne [1..%d], propusk\n",
            fname, u, v, g->n);
        continue;
    }
    if (u == v) {
        fprintf(stderr, "%s: petlya (%d,%d), propusk\n", fname, u,
v);
        continue;
    }
    add_edge(g, u, v);
}
fclose(f);
return 1;
}
// Освобождение
static void free_graph(Graph* g)
{
    for (int i = 1; i <= g->n; i++) {
        Edge* e = g->adj[i];
        while (e) { Edge* nx = e->next; free(e); e = nx; }
    }
    free(g->adj);
    g->adj = NULL;
    g->n = g->m = 0;
}
// ВСПОМОГАТЕЛЬНЫЕ
// Степень вершины
static int degree(const Graph* g, int u)
{
    int d = 0;
    for (const Edge* e = g->adj[u]; e; e = e->next) d++;
    return d;
}
// Смежны ли u и v
static int is_adj(const Graph* g, int u, int v)
{
    for (const Edge* e = g->adj[u]; e; e = e->next)
        if (e->v == v) return 1;
    return 0;
}
// ПРОВЕРКА СВЯЗНОСТИ — нужна для критерия Эйлера
// Изолированные вершины игнорируются.
static int is_connected(const Graph* g, int* visited, int* stack)
{
    if (g->n == 0) return 1;
    for (int i = 1; i <= g->n; i++) visited[i] = 0;
    int start = 1;
    for (int i = 1; i <= g->n; i++) if (g->adj[i]) { start = i; break; }
    int top = 0;
    stack[top++] = start;
    visited[start] = 1;
    while (top > 0) {
        int u = stack[--top];
        for (const Edge* e = g->adj[u]; e; e = e->next)
            if (!visited[e->v]) { visited[e->v] = 1; stack[top++] = e-
>v; }
    }
    for (int i = 1; i <= g->n; i++)
        if (g->adj[i] && !visited[i]) return 0;
}

```

```

        return 1;
    }
    // ЭЙЛЕРОВ ЦИКЛ
    // Критерий: связный + все степени чётные
    static int has_euler_circuit(const Graph* g, int* visited, int* stack)
    {
        if (g->n == 0 || g->m == 0) return 0;
        if (!is_connected(g, visited, stack)) return 0;
        for (int i = 1; i <= g->n; i++)
            if (degree(g, i) % 2 != 0) return 0;
        return 1;
    }
    // Нахождение эйлера цикла — алгоритм Хирхольцера (итеративный).
    // Возвращает длину цикла (число вершин в path), либо 0.
    static int euler_circuit(const Graph* g, int* path)
    {
        if (g->m == 0) return 0;
        int N = g->n, M = g->m;
        // Соберём рёбра один раз
        int* eu = (int*)malloc((M + 1) * sizeof(int));
        int* ev = (int*)malloc((M + 1) * sizeof(int));
        int idx = 0;
        for (int u = 1; u <= N; u++)
            for (const Edge* e = g->adj[u]; e; e = e->next)
                if (u < e->v) { idx++; eu[idx] = u; ev[idx] = e->v; }
        int* used = (int*)calloc(M + 1, sizeof(int));
        int* stack = (int*)malloc((2 * M + 2) * sizeof(int));
        int* cycle = (int*)malloc((M + 1) * sizeof(int));
        int top = 0, clen = 0;
        stack[top++] = 1;
        while (top > 0) {
            int u = stack[top - 1];
            int found = -1;
            for (int e = 1; e <= M; e++) {
                if (used[e]) continue;
                if (eu[e] == u || ev[e] == u) { found = e; break; }
            }
            if (found == -1) {
                top--;
                cycle[clen++] = u;
            }
            else {
                used[found] = 1;
                int nxt = (eu[found] == u) ? ev[found] : eu[found];
                stack[top++] = nxt;
            }
        }
        // Разворачиваем
        for (int i = 0; i < clen / 2; i++) {
            int t = cycle[i]; cycle[i] = cycle[clen - 1 - i]; cycle[clen - 1 - i] = t;
        }
        for (int i = 0; i < clen; i++) path[i] = cycle[i];

        free(eu); free(ev); free(used); free(stack); free(cycle);
        return clen;
    }
    /* ГАМИЛЬТОНОВ ЦИКЛ — итеративный перебор с возвратом
    Без рекурсии: свой стек откатов.

```

Идея:

- path[0..pos-1] – текущий путь;
 - used[] – какие вершины в пути;
 - iter[u] – указатель на "текущего соседа" в списке adj[u].
- Когда возвращаемся к u и берём следующего соседа – iter[u]++.
- Стек "возвратов": в нём вершины, из которых мы ушли вглубь.

Когда путь дошёл до длины n и последняя смежна с первой – успех. */

```
static int hamiltonian_circuit(const Graph* g, int* path)
{
    if (g->n < 3) return 0;
    int N = g->n;
    int* used = (int*)calloc(N + 1, sizeof(int));
    /* iter[u] – указатель на текущий узел списка adj[u].
       Храним как указатель на Edge* – так не надо пересчитывать. */
    Edge** iter = (Edge**)calloc(N + 1, sizeof(Edge*));
    // Стек отката – вершины текущего пути. pos = длина пути.
    path[0] = 1;
    used[1] = 1;
    int pos = 1;
    iter[1] = g->adj[1];
    int found = 0;
    while (pos > 0) {
        // Если прошли все вершины – проверим замыкание
        if (pos == N) {
            if (is_adj(g, path[pos - 1], path[0])) { found = 1; break; }
            // Нет замыкания – откат
            pos--;
            used[path[pos]] = 0;
            continue;
        }
        int u = path[pos - 1];
        Edge* e = iter[u];
        // Ищем следующего непосещённого соседа
        while (e && used[e->v]) e = e->next;
        iter[u] = e;
        if (e == NULL) {
            // У вершины u больше нет вариантов – откат
            used[u] = 0;
            iter[u] = NULL;
            pos--;
            continue;
        }
        // Берём соседа e->v, идём вглубь
        int v = e->v;
        iter[u] = e->next;    // следующий раз начнём со следующего узла
        path[pos] = v;
        used[v] = 1;
        iter[v] = g->adj[v];
        pos++;
    }
    free(iter);
    free(used);
    return found;
}

int main(int argc, char** argv)
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s <graph-file>\n", argv[0]);
        return 1;
    }
}
```

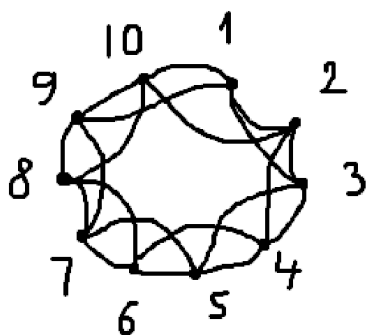
```

}
Graph g;
if (!read_graph(argv[1], &g)) return 1;
int* visited = (int*)malloc((g.n + 1) * sizeof(int));
int* buf = (int*)malloc((g.n + 1) * sizeof(int));
if (!visited || !buf) {
    free(visited); free(buf); free_graph(&g); return 1;
}
printf("Fayl: %s\n", argv[1]);
printf("Vershin: %d, ryober: %d\n\n", g.n, g.m);
// ЭЙЛЕР
printf("=== EYLEROV CIKL ===\n");
if (has_euler_circuit(&g, visited, buf)) {
    printf("Kriteriy vpolnen: graf svyaznyy, vse stepeni
chetnye.\n");
    int* path = (int*)malloc((g.m + 2) * sizeof(int));
    int len = euler_circuit(&g, path);
    printf("Eylerov cikl (%d vershin): ", len);
    for (int i = 0; i < len; i++) printf("%d ", path[i]);
    printf("\n");
    free(path);
}
else {
    printf("Eylerova cikla net.\n");
    if (!is_connected(&g, visited, buf))
        printf(" Prichina: graf nesvyaznyy.\n");
    else {
        int odd = 0;
        for (int i = 1; i <= g.n; i++)
            if (degree(&g, i) % 2 != 0) odd++;
        if (odd > 0)
            printf(" Prichina: %d vershin s nechetnoy stepenyu.\n",
odd);
    }
}
printf("\n");
// ГАМИЛЬТОН
printf("=== GAMILTONOV CIKL ===\n");
int* hpath = (int*)malloc((g.n + 1) * sizeof(int));
if (hamiltonian_circuit(&g, hpath)) {
    printf("Gamiltonov cikl (%d vershin): ", g.n);
    for (int i = 0; i < g.n; i++) printf("%d ", hpath[i]);
    printf("%d\n", hpath[0]);
}
else {
    printf("Gamiltonova cikla net.\n");
}
free(hpath);
free(visited);
free(buf);
free_graph(&g);
return 0;
}

```

Проверки:

Тест101 - euler01_cirkulyant_10vershin

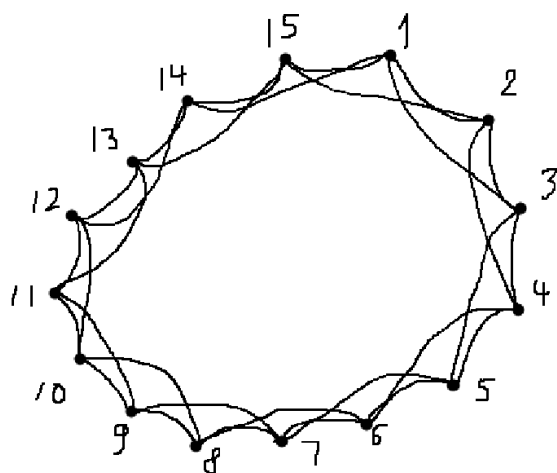


```
Fayl: tests\test101.txt
Vershin: 10, ryober: 20

=== EYLEROV CIKL ===
Kriteriy vpolnen: graf svyaznyy, vse stepeni chetnye.
Eulerov cikl (21 vershin): 1 10 2 1 9 7 5 3 2 4 6 8 10 9 8 7 6 5 4 3 1

=== GAMILTONOV CIKL ===
Gamiltonov cikl (10 vershin): 1 10 2 4 6 8 9 7 5 3 1
```

Tecr102 - eyler02_cirkulyant_15vershin

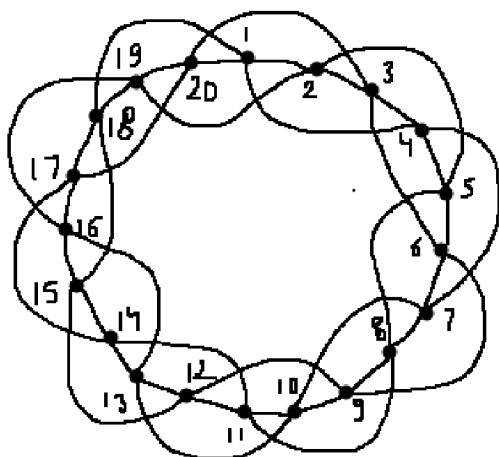


```
Fayl: tests\test102.txt
Vershin: 15, ryober: 30

=== EYLEROV CIKL ===
Kriteriy vpolnen: graf svyaznyy, vse stepeni chetnye.
Eulerov cikl (31 vershin): 1 15 2 1 14 12 10 8 6 4 2 3 5 7 9 11 13 15 14
13 12 11 10 9 8 7 6 5 4 3 1

=== GAMILTONOV CIKL ===
Gamiltonov cikl (15 vershin): 1 15 2 4 6 8 10 12 14 13 11 9 7 5 3 1
```

Tecr103 - eyler03_cirkulyant_20vershin



```
Fayl: tests\test103.txt
Vershin: 20, ryober: 40
```

=== EYLEROV CIKL ===

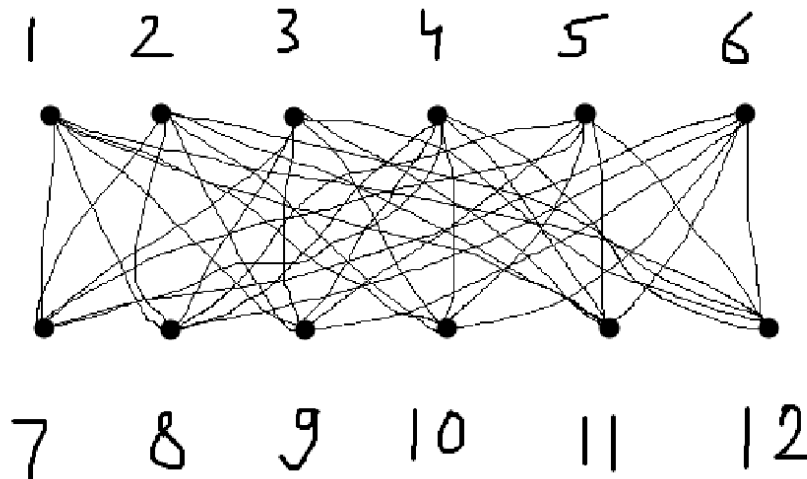
Kriteriy vypolnen: graf svyaznyy, vse stepeni chetnye.

Eylerov cikel (41 vershin): 1 20 3 2 1 18 15 12 9 6 3 4 7 6 5 2 19 16 13 1
0 7 8 11 14 17 20 19 18 17 16 15 14 13 12 11 10 9 8 5 4 1

=== GAMILTONOV CIKL ===

Gamiltonov cikel (20 vershin): 1 20 3 6 9 12 15 18 19 2 5 8 11 14 17 16 13
10 7 4 1

Tecr104 - eyler04_K66_12vershin



```
Fayl: tests\test104.txt
Vershin: 12, ryober: 36
```

=== EYLEROV CIKL ===

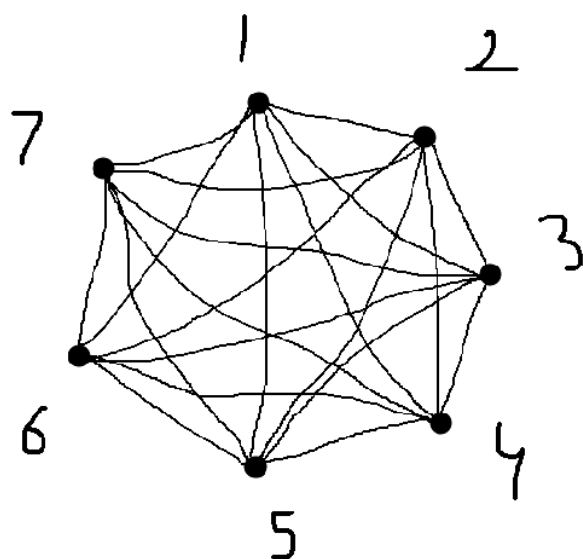
Kriteriy vypolnen: graf svyaznyy, vse stepeni chetnye.

Eylerov cikel (37 vershin): 1 12 2 11 1 10 2 9 1 8 2 7 3 12 4 11 3
10 4 9 3 8 4 7 5 12 6 11 5 10 6 9 5 8 6 7 1

=== GAMILTONOV CIKL ===

Gamiltonov cikel (12 vershin): 1 12 6 11 5 10 4 9 3 8 2 7 1

Tecr105 - eyler05_K7_7vershin



```
Fayl: tests\test105.txt
Vershin: 7, ryober: 21
```

=== EYLEROV CIKL ===

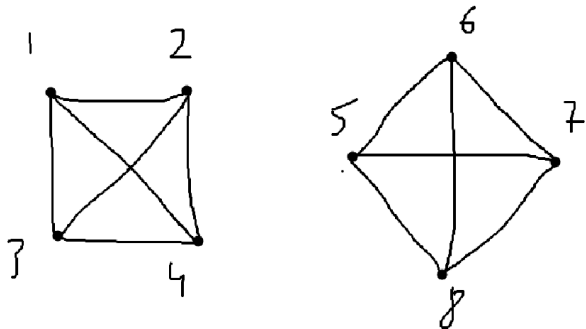
Kriteriy vpolnen: graf svyaznyy, vse stepeni chetnye.

Eylerov cikel (22 vershin): 1 7 2 1 6 2 5 1 4 2 3 7 4 3 6 4 5 7 6 5 3 1

=== GAMILTONOV CIKL ===

Gamiltonov cikel (7 vershin): 1 7 6 5 4 3 2 1

Tect106 - eyler06_dva_K4_8vershin



```
Fayl: tests\test106.txt
Vershin: 8, ryober: 12
```

=== EYLEROV CIKL ===

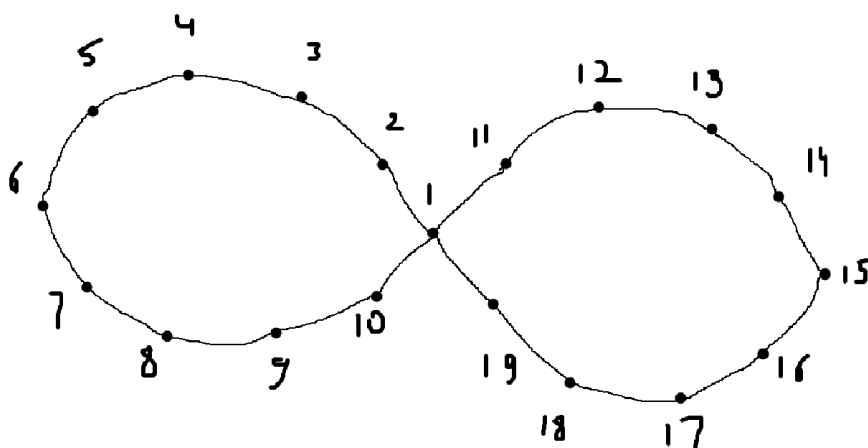
Eylerova cikla net.

Prichina: graf nesvyaznyy.

=== GAMILTONOV CIKL ===

Gamiltonova cikla net.

Tect107 - eyler07_pesochnye_chasy_19vershin



```
Fayl: tests\test107.txt
Vershin: 19, ryober: 20
```

=== EYLEROV CIKL ===

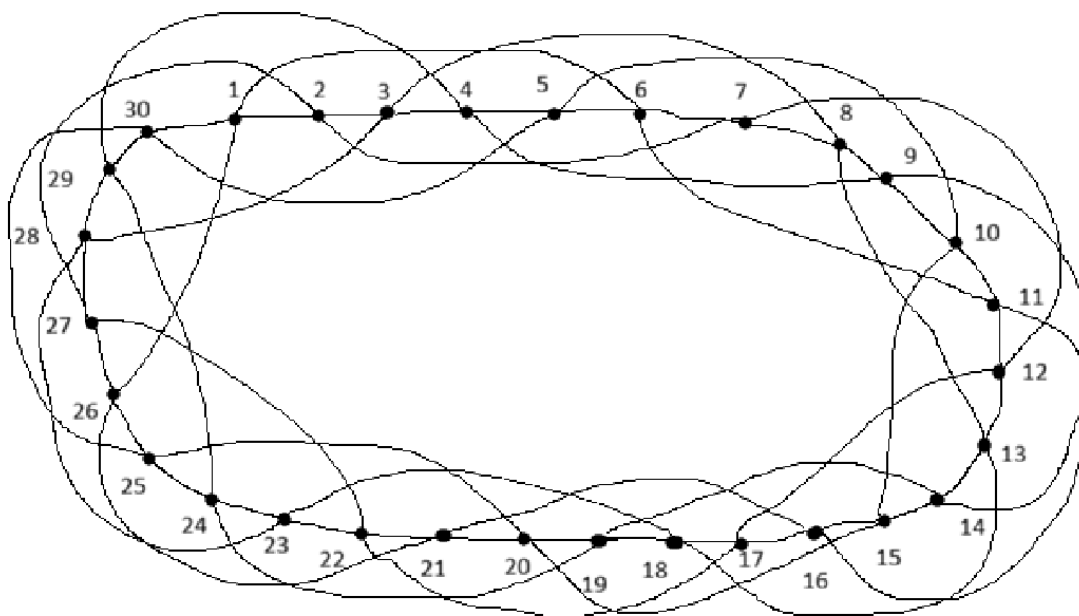
Kriteriy vpolnen: graf svyaznyy, vse stepeni chetnye.

Eylerov cikel (21 vershin): 1 19 18 17 16 15 14 13 12 11 1
10 9 8 7 6 5 4 3 2 1

=== GAMILTONOV CIKL ===

Gamiltonova cikla net.

Tect108 - eyler08_cirkulyant_30vershin



```
Fayl: tests\test108.txt
Vershin: 30, ryober: 60
```

```
=== EYLEROV CIKL ===
```

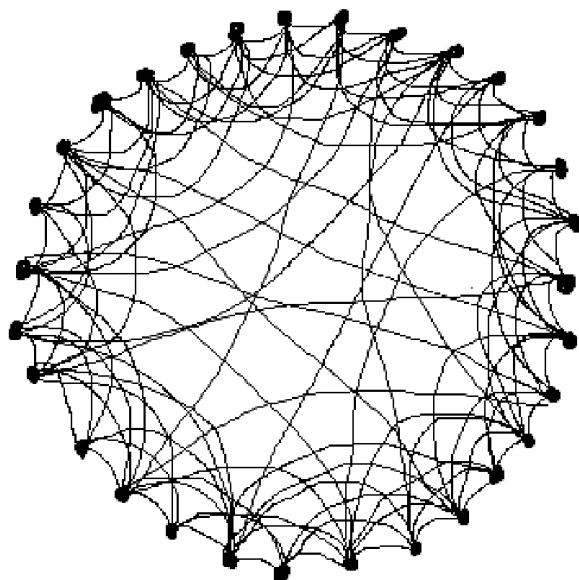
```
Kriteriy vypolnen: graf svyaznyy, vse stepeni chetnye.
```

```
Eylerov cikl (61 vershin): 1 30 5 4 3 2 1 26 21 16 11 6 5 10 9 4
29 24 19 14 9 8 3 28 23 18 13 8 7 2 27 22 17 12 11 10 15 20 25 30
29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 7 6 1
```

```
=== GAMILTONOV CIKL ===
```

```
Gamiltonov cikl (30 vershin): 1 30 5 10 15 20 25 26 27 2 7 12 17
18 19 24 29 4 9 14 13 8 3 28 23 22 21 16 11 6 1
```

Tecr109 – eyler09_K30_30vershin



```
Fayl: tests\test109.txt
Vershin: 30, ryober: 435
```

```
=== EYLEROV CIKL ===
```

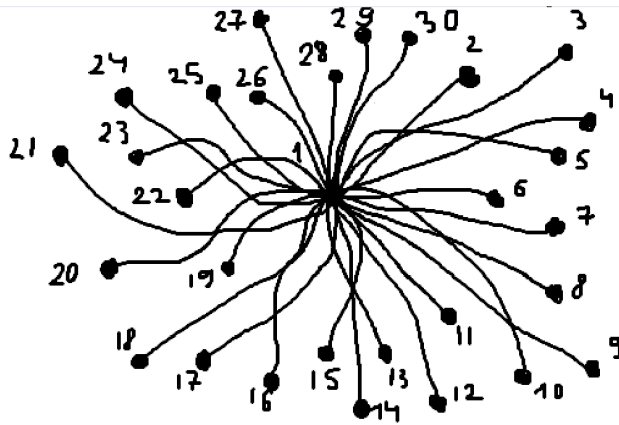
```
Eylerova cikla net.
```

```
Prichina: 30 vershin s nechetnoy stepenyu.
```

```
=== GAMILTONOV CIKL ===
```

```
Gamiltonov cikl (30 vershin): 1 30 29 28 27 26 25 24 23 22 21 20
19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
```

Тест110 – eyler10_zvezda_30vershin



Fayl: tests\test110.txt

Vershin: 30, ryober: 29

=== EYLEROV CIKL ===

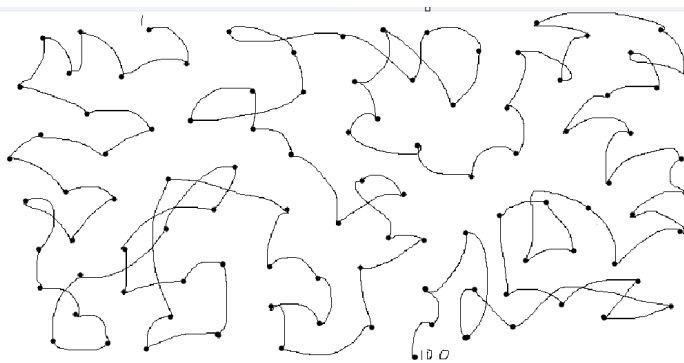
Eylerova cikla net.

Prichina: 30 vershin s nechetnoy stepenyu.

=== GAMILTONOV CIKL ===

Gamiltonova cikla net.

Тест111 – eyler11_cep_100vershin



Fayl: tests\test111.txt

Vershin: 100, ryober: 99

=== EYLEROV CIKL ===

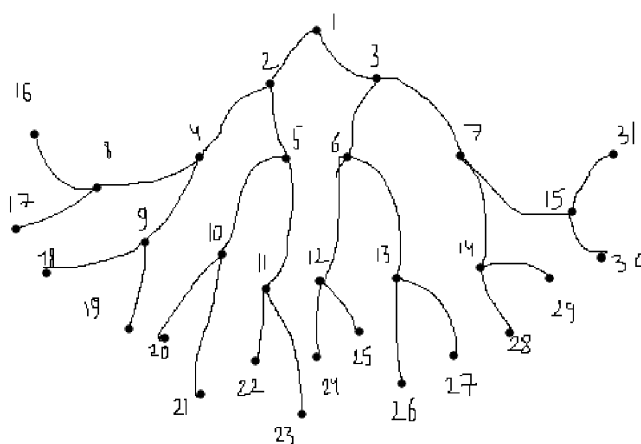
Eylerova cikla net.

Prichina: 2 vershin s nechetnoy stepenyu.

=== GAMILTONOV CIKL ===

Gamiltonova cikla net.

Тест112 – eyler12_derevo_31vershin



```
Fayl: tests\test112.txt
Vershin: 31, ryober: 30
```

```
=== EYLEROV CIKL ===
```

Eylerova cikla net.

Prichina: 30 vershin s nechetnoy stepenyu.

```
=== GAMILTONOV CIKL ===
```

Gamiltonova cikla net.

Тест113 – eyler13_cirkulyant_200vershin

$$C_{200}(1,2): \quad X = \{1, \dots, 200\}, \quad U = \{\{i, j\} : j - i \equiv \pm 1, \pm 2 \pmod{200}\}$$

```
Fayl: tests\test113.txt
Vershin: 200, ryober: 400
```

```
=== EYLEROV CIKL ===
```

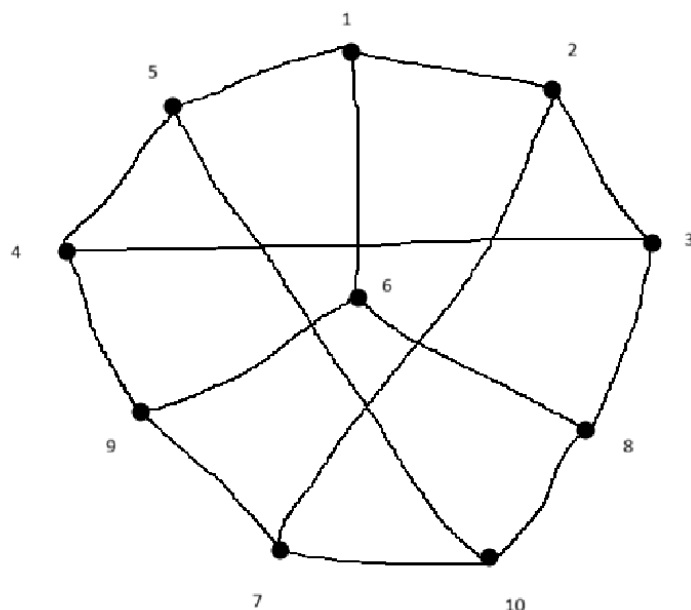
Kriteriy vypolnen: graf svyaznyy, vse stepeni chetnye.

Eylerov cikl (401 vershin): 1 200 2 1 199 197 195 193 191 189 187 185 183 181 179 177 175 173 171 169 167 165 163 161 159 157 155 153 151 149 147 145 143 141 139 137 135 133 131 129 127 125 123 12 119 117 115 113 111 109 107 105 103 101 99 97 95 93 91 89 87 85 83 81 79 77 75 73 71 69 67 65 63 61 59 57 55 53 51 49 47 45 43 41 39 37 35 33 31 29 27 25 23 21 19 17 15 13 11 9 7 5 3 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72 74 76 78 80 82 84 86 88 90 92 94 96 98 100 102 104 106 108 110 112 114 116 118 120 122 124 126 128 130 132 134 136 138 140 142 144 146 148 150 152 154 156 158 160 162 164 166 168 170 172 174 176 178 18 0 182 184 186 188 190 192 194 196 198 200 199 198 197 196 195 194 193 192 191 190 189 188 187 186 185 184 183 182 181 180 179 178 177 176 175 174 173 172 171 170 169 168 167 166 165 164 163 162 16 1 160 159 158 157 156 155 154 153 152 151 150 149 148 147 146 145 144 143 142 141 140 139 138 137 136 135 134 133 132 131 130 129 128 127 126 125 124 123 122 121 120 119 118 117 116 115 114 113 11 2 111 110 109 108 107 106 105 104 103 102 101 100 99 98 97 96 95 94 93 92 91 90 89 88 87 86 85 84 83 82 81 80 79 78 77 76 75 74 73 72 71 70 69 68 67 66 65 64 63 62 61 60 59 58 57 56 55 54 53 52 51 50 49 48 47 46 45 44 43 42 41 40 39 38 37 36 35 34 33 32 31 30 29 28 27 26 25 24 23 22 21 20 19 1 8 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 1

```
=== GAMILTONOV CIKL ===
```

Gamiltonov cikl (200 vershin): 1 200 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60 62 64 66 68 70 72 74 76 78 80 82 84 86 88 90 92 94 96 98 100 102 104 106 108 110 112 114 116 118 120 122 124 126 128 130 132 134 136 138 140 142 144 146 148 150 152 154 15 6 158 160 162 164 166 168 170 172 174 176 178 180 182 184 186 188 190 192 194 196 198 199 197 195 193 191 189 187 185 183 181 179 177 175 173 171 169 167 165 163 161 159 157 155 153 151 149 147 14 5 143 141 139 137 135 133 131 129 127 125 123 121 119 117 115 113 111 109 107 105 103 101 99 97 95 93 91 89 87 85 83 81 79 77 75 73 71 69 67 65 63 61 59 57 55 53 51 49 47 45 43 41 39 37 35 33 31 2 9 27 25 23 21 19 17 15 13 11 9 7 5 3 1

Тест114 – eyler14_Petersen_10vershin



```

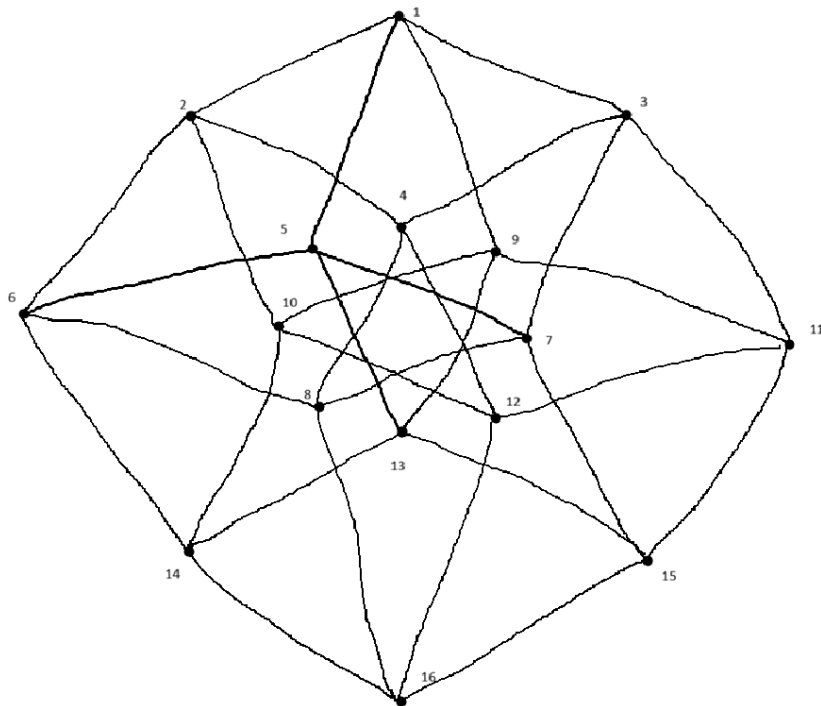
Fayl: tests\test114.txt
Vershin: 10, ryober: 15

=== EYLEROV CIKL ===
Eylerova cikla net.
Prichina: 10 vershin s nechetnoy stepenyu.

=== GAMILTONOV CIKL ===
Gamiltonova cikla net.

```

Tect115 – eyler15_giperkub_Q4_16vershin



```

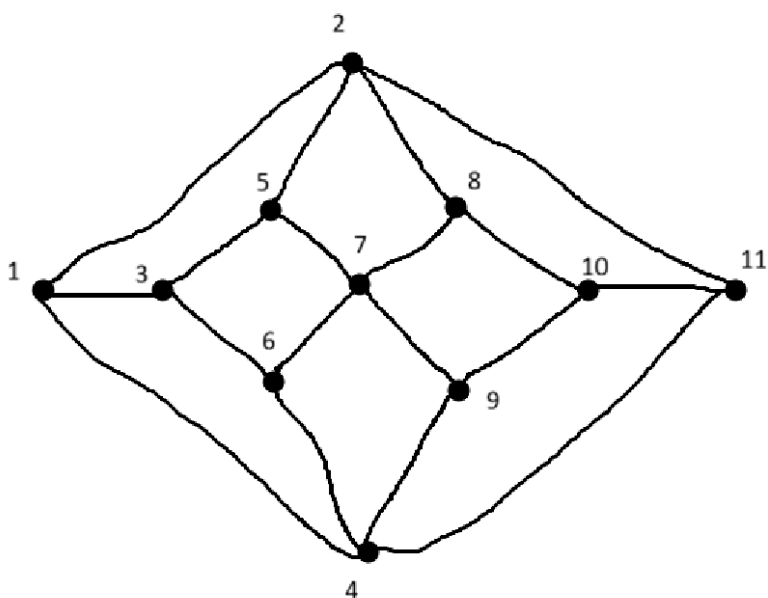
Fayl: tests\test115.txt
Vershin: 16, ryober: 32

=== EYLEROV CIKL ===
Kriteriy vpolnen: graf svyaznyy, vse stepeni chetnye.
Eylerov cikl (32 vershin): 1 9 13 5 1 3 11 9 10 2 6 5 7 3 4 12 10 14 6
8 7 15 11 12 16 14 13 15 16 8 4 2 1

=== GAMILTONOV CIKL ===
Gamiltonov cikl (16 vershin): 1 9 13 15 16 14 10 12 11 3 7 8 4 2 6 5 1

```

Tect201 – gam01_Herschel_11vershin



Fayl: tests\test201.txt
Vershin: 11, ryober: 18

=== EYLEROV CIKL ===

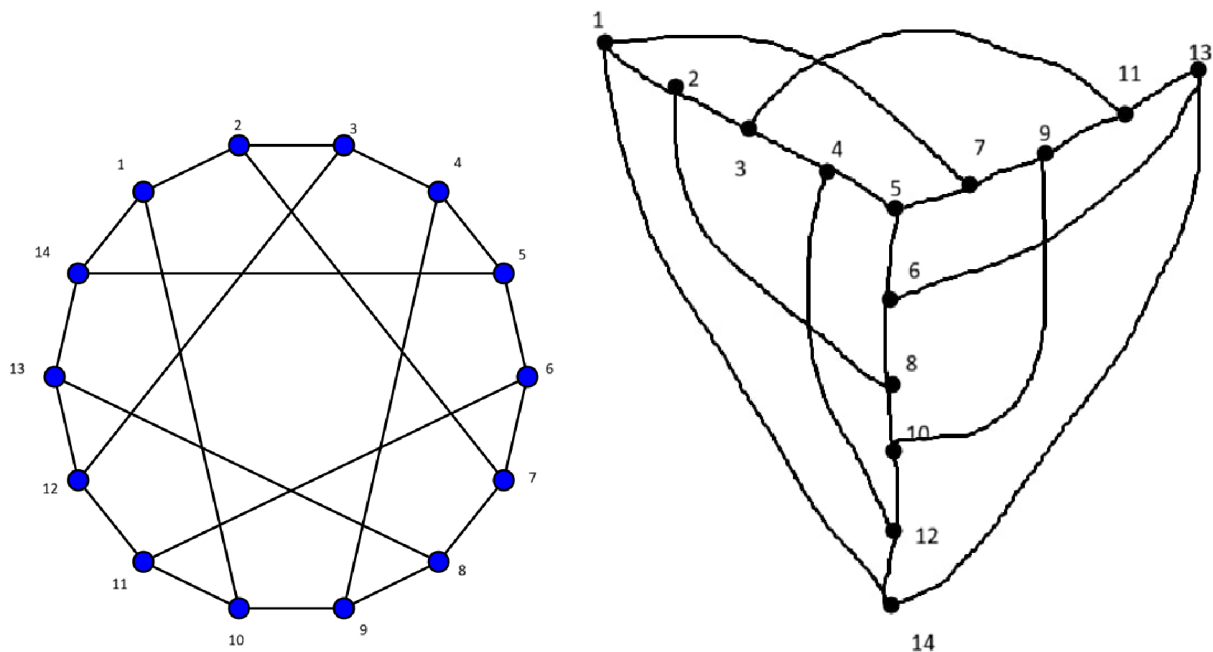
Eylerova cikla net.

Prichina: 8 vershin s nechetnoy stepenyu.

=== GAMILTONOV CIKL ===

Gamiltonova cikla net.

Tect202 – gam02_Hivud_14vershin



Fayl: tests\test202.txt
Vershin: 14, ryober: 21

=== EYLEROV CIKL ===

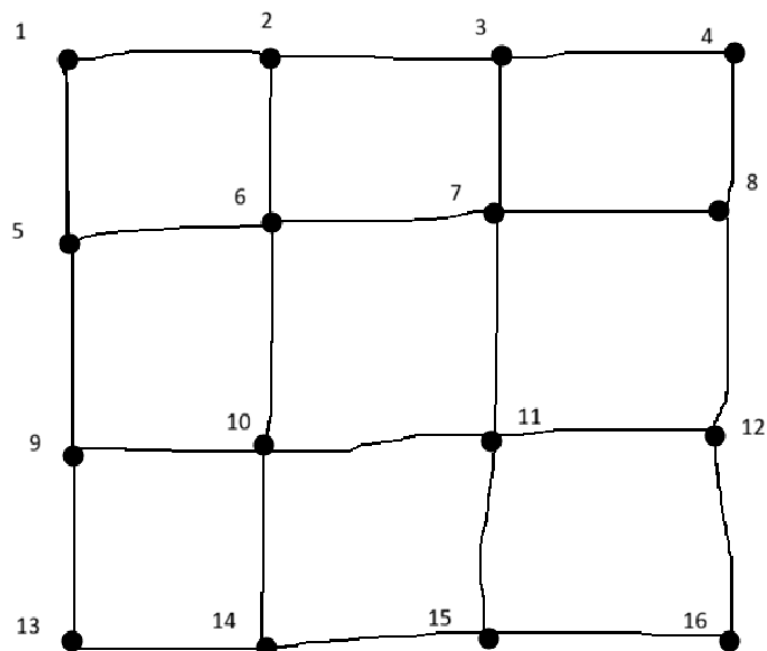
Eylerova cikla net.

Prichina: 14 vershin s nechetnoy stepenyu.

=== GAMILTONOV CIKL ===

Gamiltonov cikl (14 vershin): 1 14 13 11 9 10 12 4 3 2 8 6 5 7 1

Tect203 – gam03_reshetka_4x4_16vershin



```
Fayl: tests\test203.txt
Vershin: 16, ryober: 24

=== EYLEROV CIKL ===
Eylerova cikla net.
Prichina: 8 vershin s nechetnoy stepenyu.

=== GAMILTONOV CIKL ===
Gamiltonov cikl (16 vershin): 1 5 9 13 14 15 16 12 11 10 6 7 8 4 3 2 1
```

Тест204 – gam05_reshetka_10x10_100vershin

$$P_{10} \square P_{10}$$

```
Fayl: tests\test204.txt
Vershin: 100, ryober: 180

=== EYLEROV CIKL ===
Eylerova cikla net.
Prichina: 32 vershin s nechetnoy stepenyu.

=== GAMILTONOV CIKL ===
Gamiltonov cikl (100 vershin): 1 11 21 31 41 51 61 71 81 91 92 93 94 95 96 97 98 99 10
0 90 89 88 87 86 85 84 83 82 72 73 74 75 76 77 78 79 80 70 69 68 67 66 65 64 63 62 52
53 54 55 56 57 58 59 60 50 49 48 47 46 45 44 43 42 32 33 34 35 36 37 38 39 40 30 29 28
27 26 25 24 23 22 12 13 14 15 16 17 18 19 20 10 9 8 7 6 5 4 3 2 1
```

Тест205 – gam05_reshetka_100x10_1000vershin

$$P_{100} \square P_{10}$$

```
Fayl: tests\test205.txt
Vershin: 1000, ryober: 1890

=== EYLEROV CIKL ===
Eylerova cikla net.
Prichina: 212 vershin s nechetnoy stepenyu.

=== GAMILTONOV CIKL ===
Gamiltonov cikl (1000 vershin): 1 101 201 301 401 501 601 701 801 901 902 903 904 905 906 907 908 909 910 911 912 913 91
4 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 94
4 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 97
4 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 900 899 898 8
97 896 895 894 893 892 891 890 889 888 887 886 885 884 883 882 881 880 879 878 877 876 875 874 873 872 871 870 869 868 8
67 866 865 864 863 862 861 860 859 858 857 856 855 854 853 852 851 850 849 848 847 846 845 844 843 842 841 840 839 838 8
37 836 835 834 833 832 831 830 829 828 827 826 825 824 823 822 821 820 819 818 817 816 815 814 813 812 811 810 809 808 8
07 806 805 804 803 802 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 7
26 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 7
56 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 7
86 787 788 789 790 791 792 793 794 795 796 797 798 799 800 700 699 698 697 696 695 694 693 692 691 690 689 688 687 686 6
85 684 683 682 681 680 679 678 677 676 675 674 673 672 671 670 669 668 667 666 665 664 663 662 661 660 659 658 657 656 6
55 654 653 652 651 650 649 648 647 646 645 644 643 642 641 640 639 638 637 636 635 634 633 632 631 630 629 628 627 626 6
25 624 623 622 621 620 619 618 617 616 615 614 613 612 611 610 609 608 607 606 605 604 603 602 502 503 504 505 506 507 5
08 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 5
38 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 5
68 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 5
98 599 600 500 499 498 497 496 495 494 493 492 491 490 489 488 487 486 485 484 483 482 481 480 479 478 477 476 475 474 4
73 472 471 470 469 468 467 466 465 464 463 462 461 460 459 458 457 456 455 454 453 452 451 450 449 448 447 446 445 444 4
43 442 441 440 439 438 437 436 435 434 433 432 431 430 429 428 427 426 425 424 423 422 421 420 419 418 417 416 415 414 4
13 412 411 410 409 408 407 406 405 404 403 402 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 3
20 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 3
50 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 3
80 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 300 299 298 297 296 295 294 293 292 2
91 290 289 288 287 286 285 284 283 282 281 280 279 278 277 276 275 274 273 272 271 270 269 268 267 266 265 264 263 262 2
61 260 259 258 257 256 255 254 253 252 251 250 249 248 247 246 245 244 243 242 241 240 239 238 237 236 235 234 233 232 2
31 230 229 228 227 226 225 224 223 222 221 220 219 218 217 216 215 214 213 212 211 210 209 208 207 206 205 204 203 202 1
02 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 1
32 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 1
62 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 1
92 193 194 195 196 197 198 199 200 100 99 98 97 96 95 94 93 92 91 90 89 88 87 86 85 84 83 82 81 80 79 78 77 76 75 74 73
72 71 70 69 68 67 66 65 64 63 62 61 60 59 58 57 56 55 54 53 52 51 50 49 48 47 46 45 44 43 42 41 40 39 38 37 36 35 34 33
32 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
```

Вывод: реализована программа на С для поиска эйлерова и гамильтонова циклов в неориентированном графе. Проверена на 20 тестах — от малых графов до графа с 1000 вершинами. Результаты совпали с теорией.